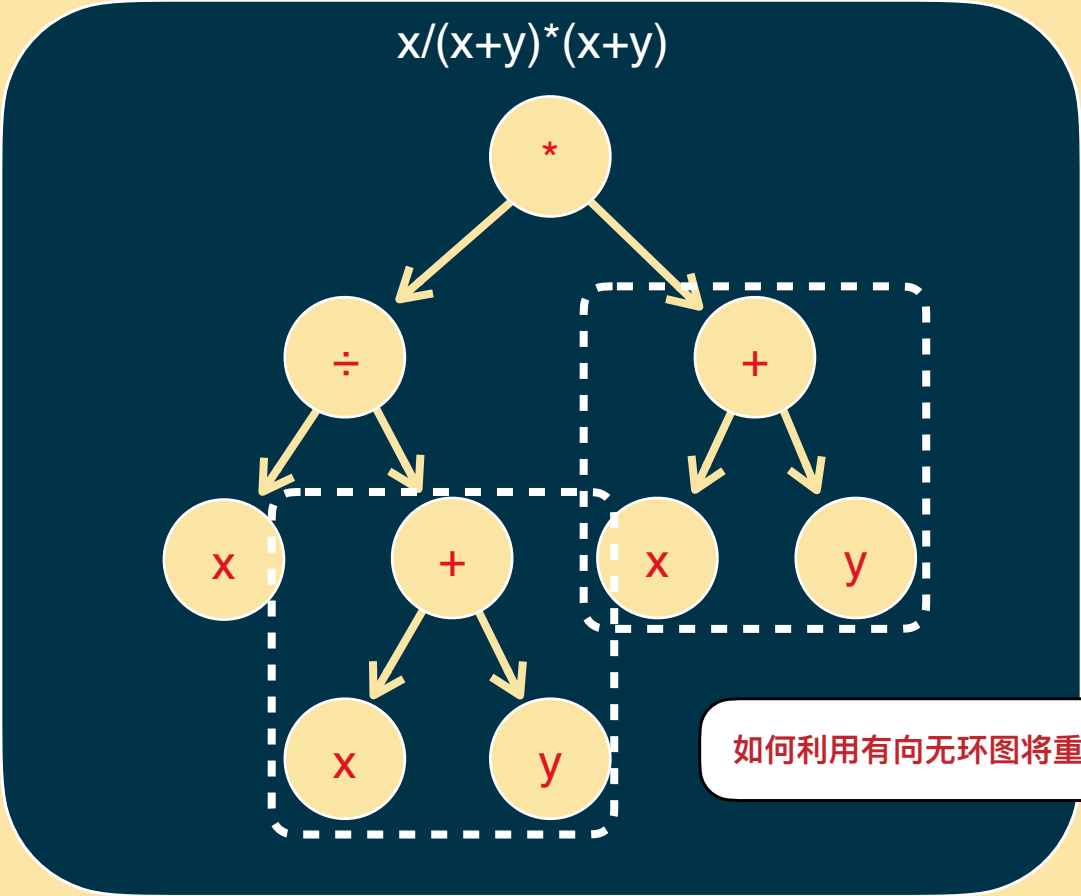




3. 有向无环图 (DAG图) 描述表达式

DAG图：不存在环的有向图





3. 有向无环图 (DAG图) 描述表达式

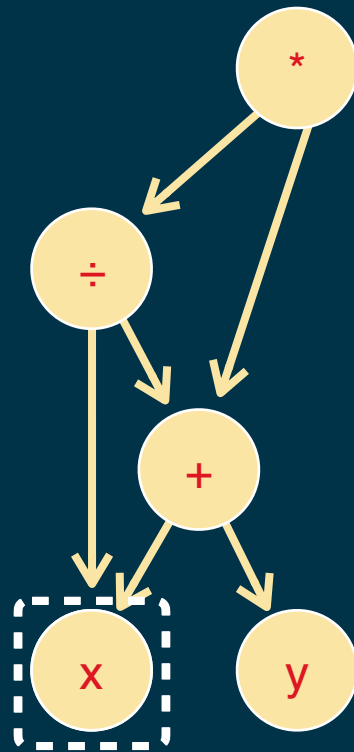
DAG图：不存在环的有向图

如何利用有向无环图将重复部分化简？

1. 不出现重复的操作数顶点

2. 尽可能化简

$x/(x+y)*(x+y)$



化简完毕



4. 拓扑排序

概念

关键路径

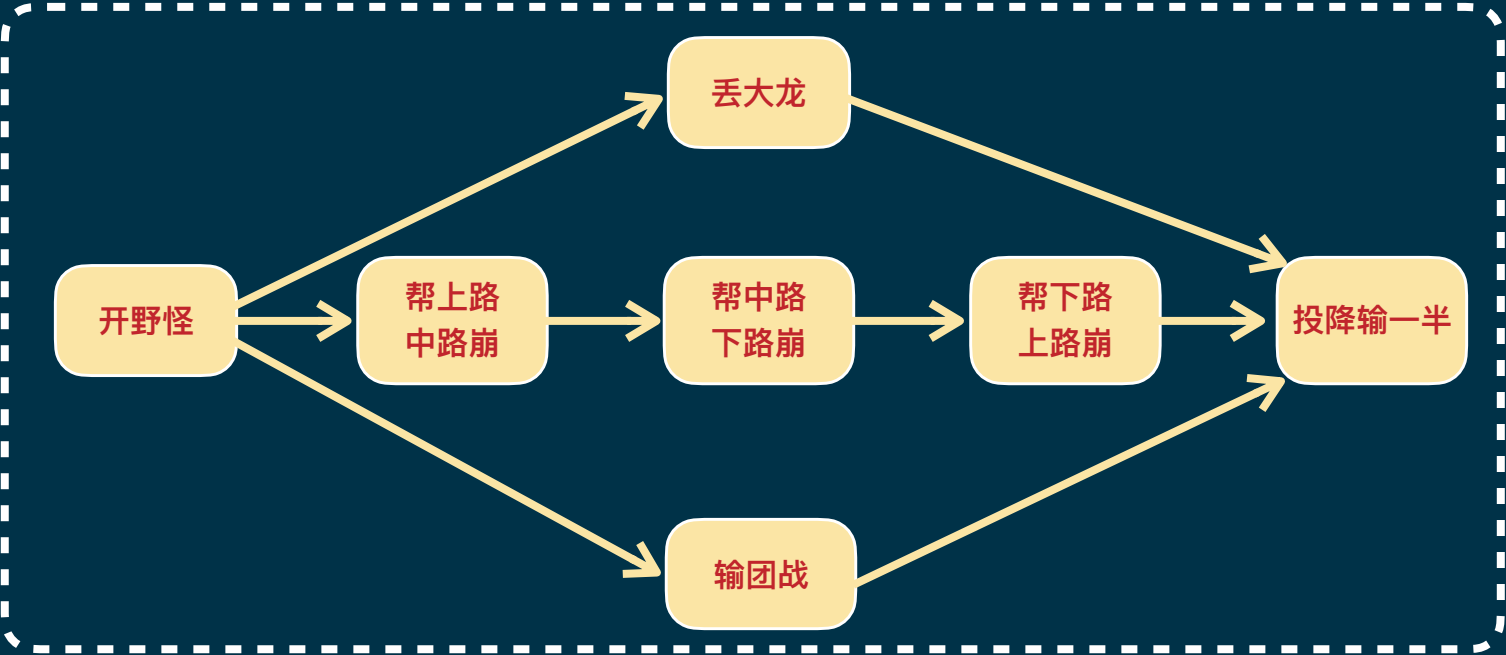


4. 拓扑排序

概念

关键路径

AOV网：用有向无环图表示一个工程，每个顶点代表一个活动，并用有向边表示活动[开野怪]必须先于活动[帮上路]进行的关系，则称这种有向图为 顶点表示活动的网络，简称 **AOV 网**。

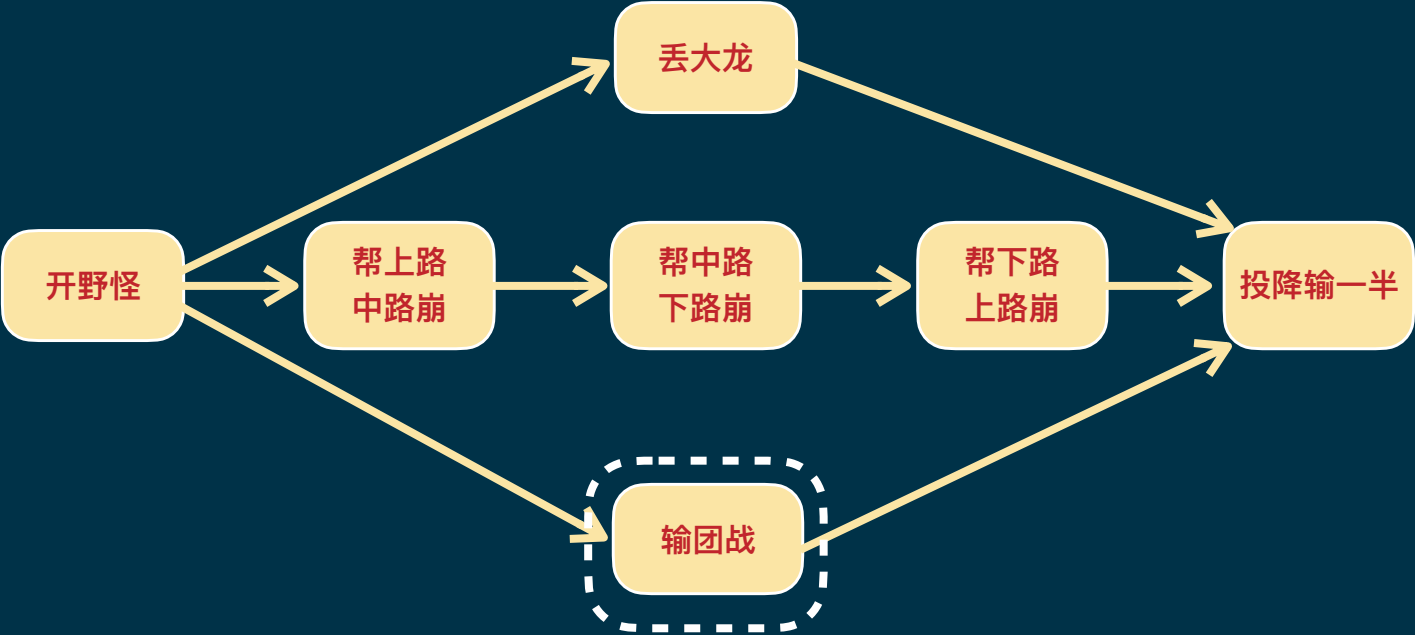




4. 拓扑排序

概念

关键路径



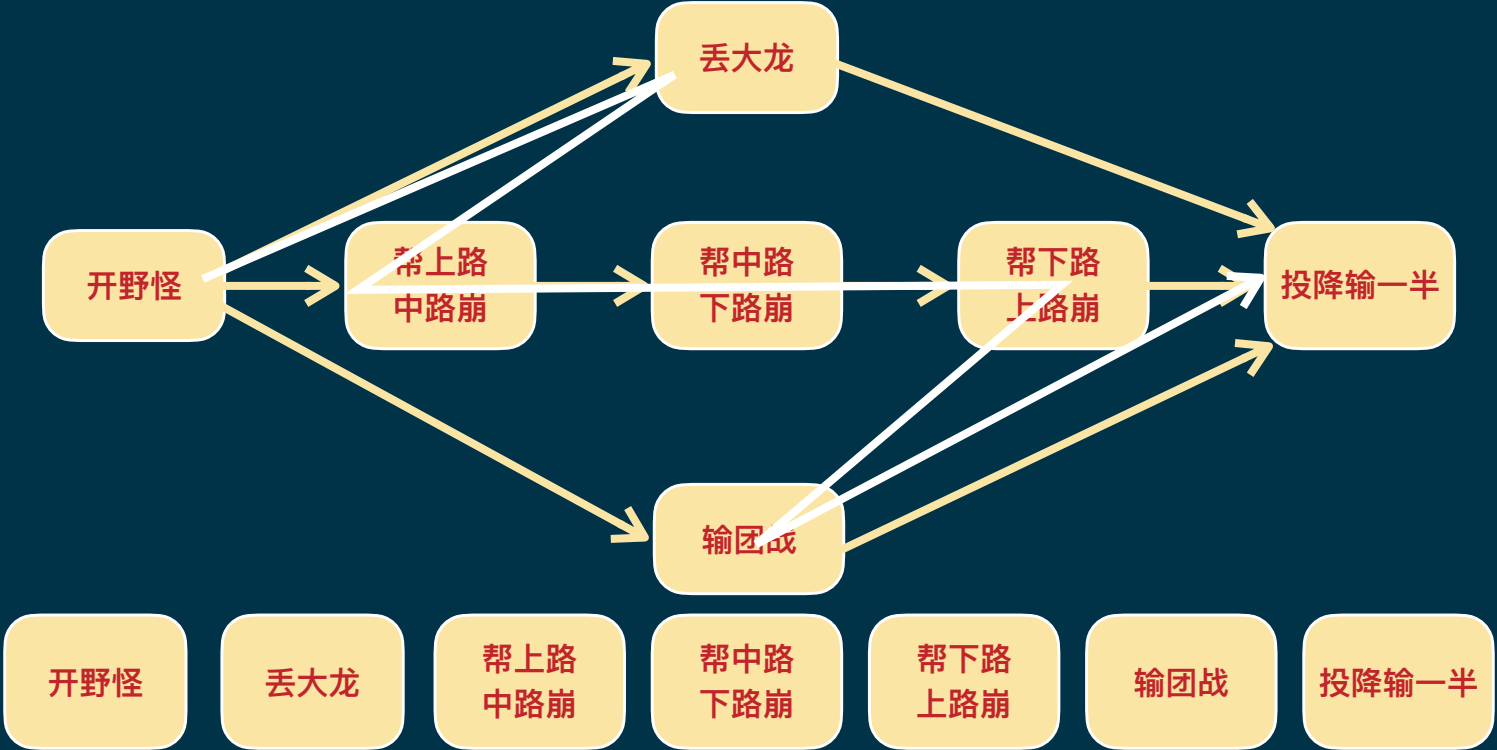
图中的每个顶点被称为活动



4. 拓扑排序

概念

关键路径

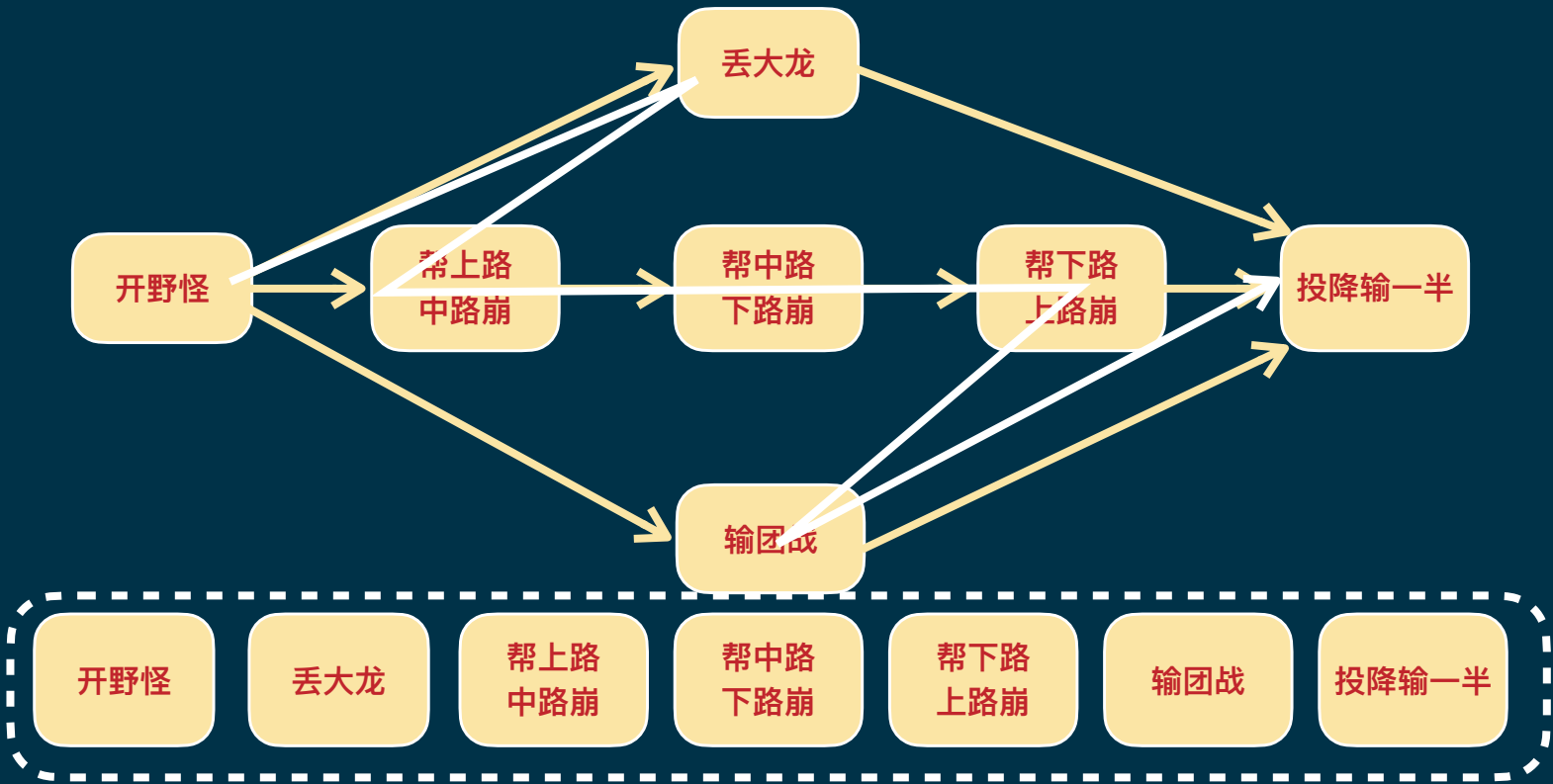




4. 拓扑排序

概念

关键路径



拓扑排序指一个序列，每个顶点只出现一次，并且严格按照箭头先后顺序
一个aov网可能有多个拓扑排序序列



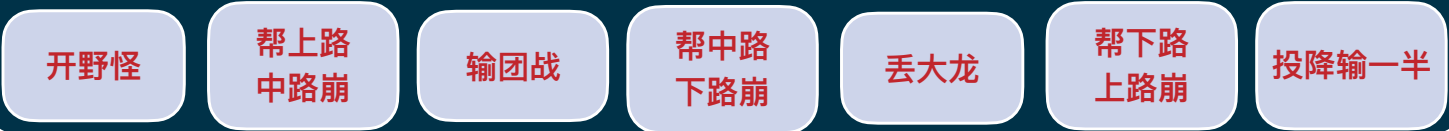
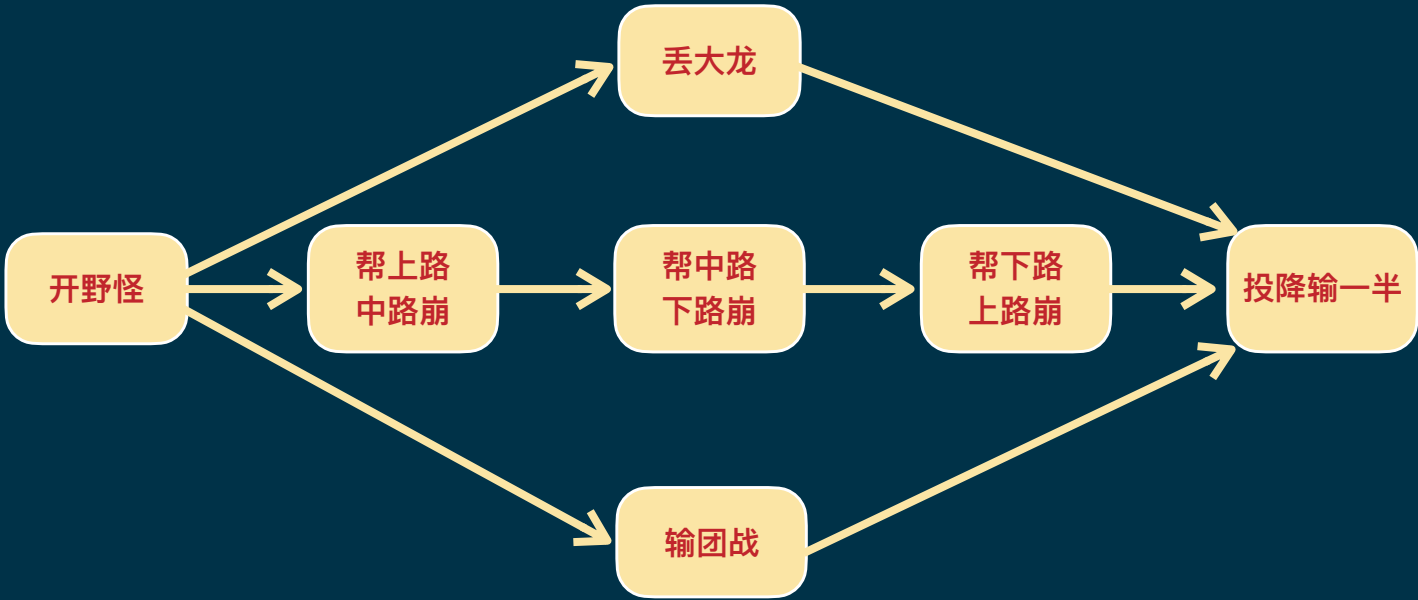
4. 拓扑排序

概念

关键路径

拓扑排序方法步骤

- 1. 在有向图中选择一个无前驱的顶点并输出它
- 2. 从图中删除该顶点和所有从它发射出去的弧



4. 拓扑排序

概念

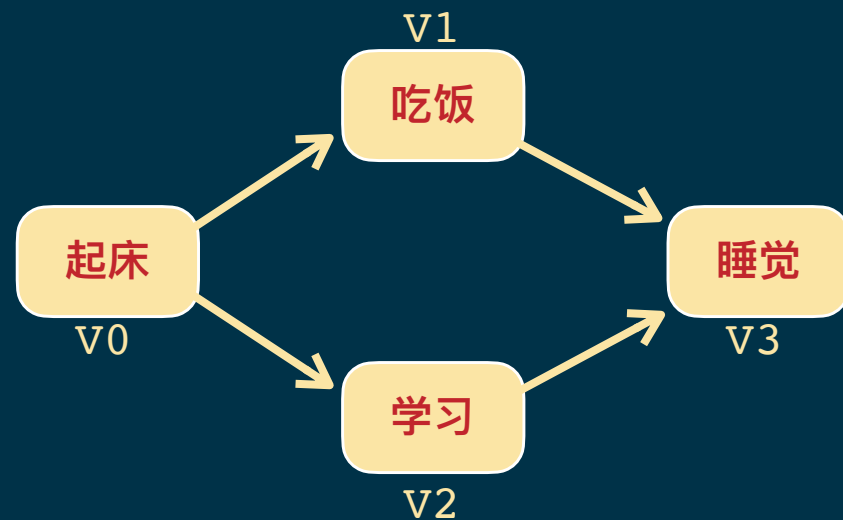
关键路径

BOK数据结构



```
bool visited (MVNUM];  
// 对图G进行深度优先遍历  
void DFSTraverse(Graph G) {  
    // 初始化所有顶点的访问标记为false (未访问)  
    for(v=0; v<G.vexnum; ++v)  
        visited [v]=false;  
    time=0; // 时间戳初始化为0  
    for (v=0; v<G. vexnum; ++v) // 遍历图中的每个顶点  
        //如果该顶点未被访问, 从该顶点开始进行深度优先搜索  
        if(!visited [v]) DFS(G,v);  
}
```

```
void DFS (Graph G, int v){  
    //标记当前顶点v为已访问  
    visited [v]=TRUE;  
    // 访问当前顶点v  
    visit(v);  
    //遍历v的所有邻接顶点  
    for(w=FirstNeighbor(G,v);w>=0; w=NextNeighbor(G,v,w)){  
        //如果邻接顶点未被访问, 递归地对其进行深度优先搜索  
        if(!visited[w]) DFS(G,w);  
    }  
    //遍历完成后, 增加时间戳, 并记录顶点v的完成时间  
    time=time+1;  
    finishTime[v]=time;  
}
```



finishTime[4] 4 2 3 1
 V0 V1 V2 V3

将finishTime数组按照元素降序输出下标即为拓扑排序序列
(V0, V2, V1, V3)

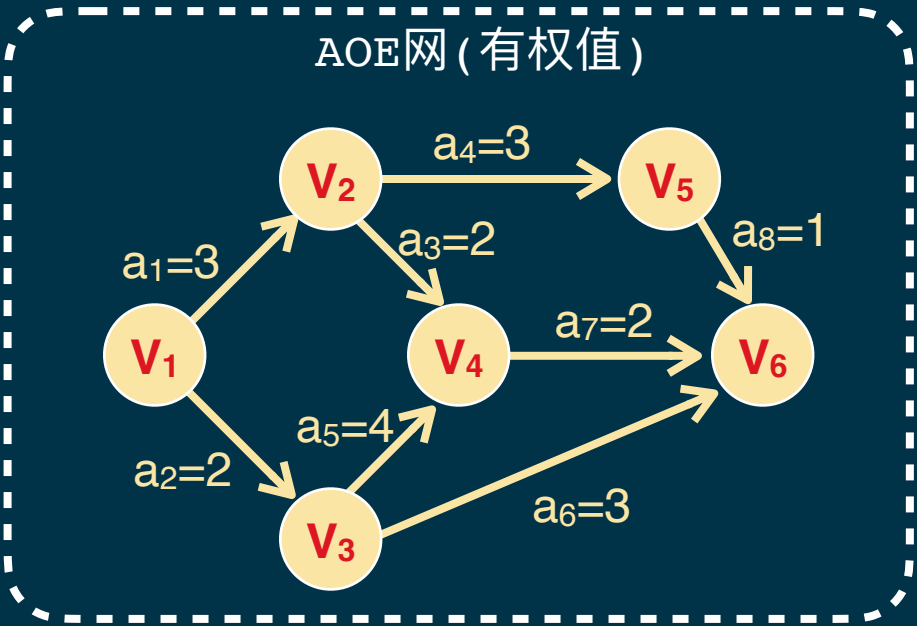
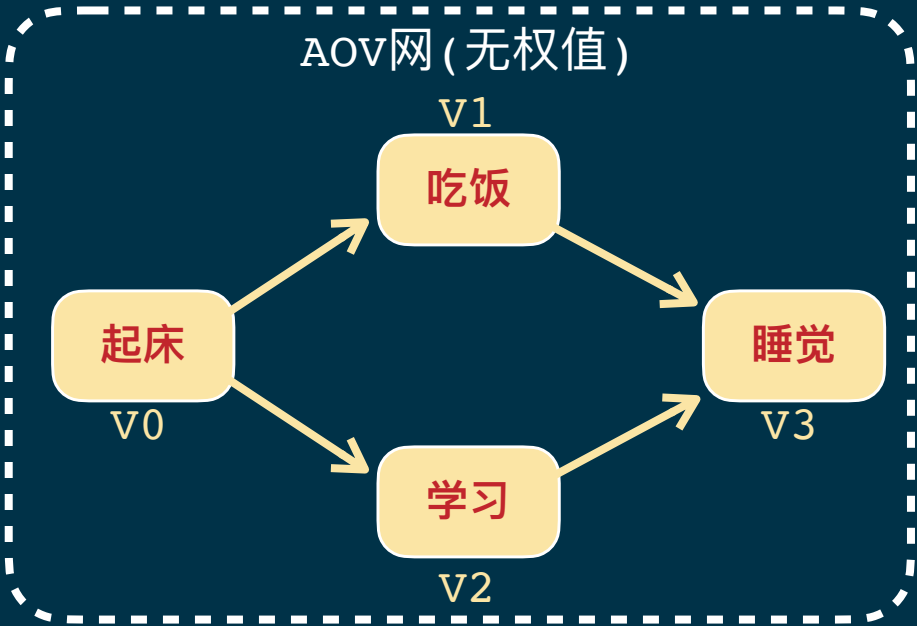
visited[4] T T T T



4. 拓扑排序

概念

关键路径



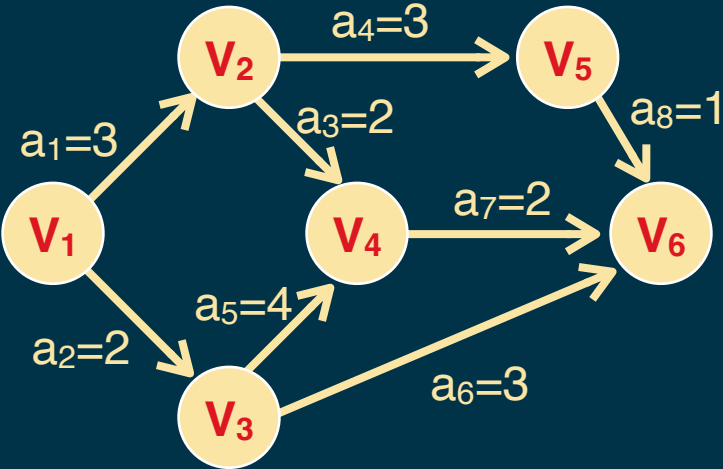


4. 拓扑排序

概念

关键路径

AOE网 (有权值)



顶点用来表示事件
边用来表示活动
(Activity On Edge, 活动在边上)



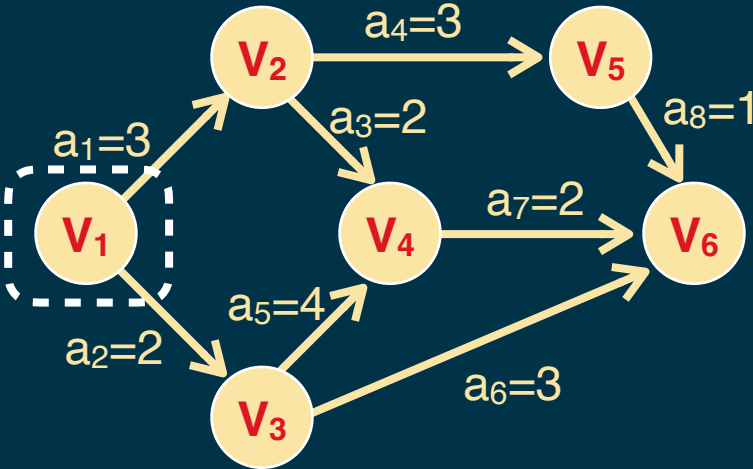
4. 拓扑排序

概念

关键路径

AOE网 (有权值)

开始顶点 (源点) :
AOE网中通常仅有一个入度为0的顶点





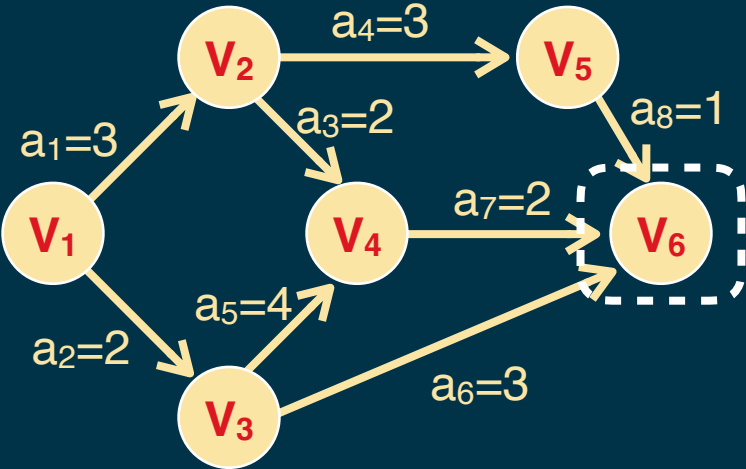
4. 拓扑排序

概念

关键路径

AOE网 (有权值)

开始顶点 (源点) :
AOE网中通常仅有一个入度为0的顶点



结束顶点 (汇点) :
AOE网中通常仅有一个出度为0的顶点

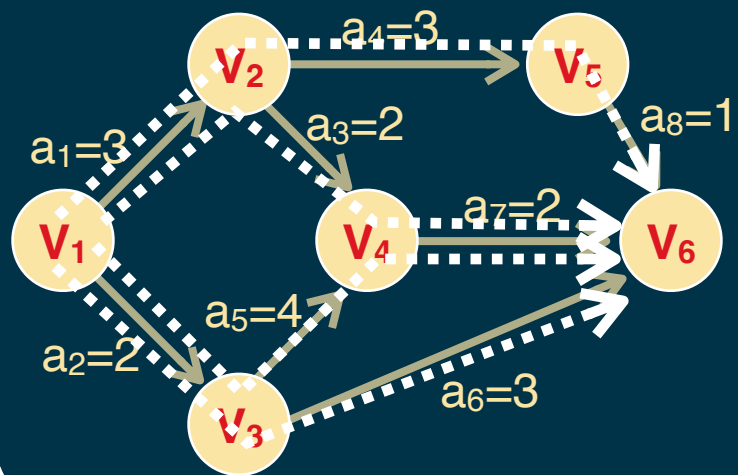


4. 拓扑排序

概念

关键路径

AOE网 (有权值)



开始顶点 (源点):
AOE网中通常仅有一个入度为0的顶点

结束顶点 (汇点):
AOE网中通常仅有一个出度为0的顶点

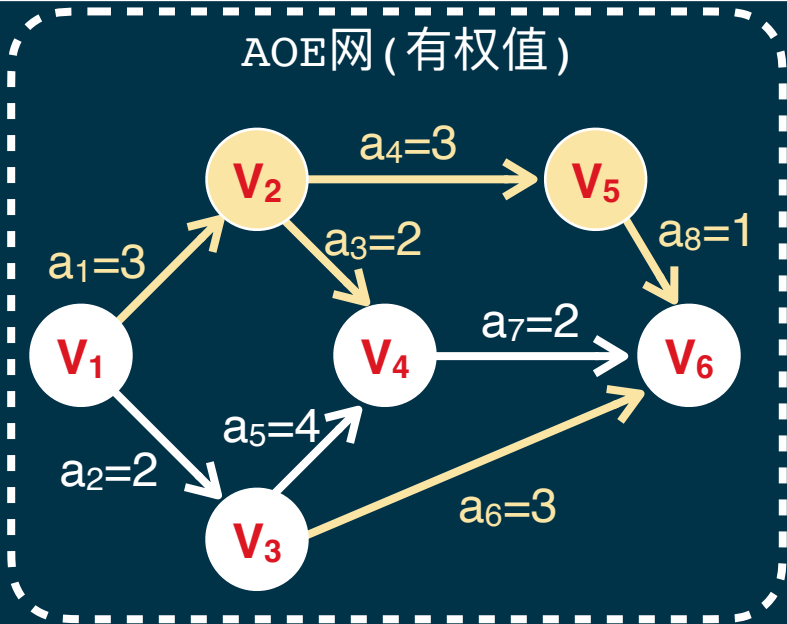
从源点到汇点可能会有很多条路, 长度 (权值和) 不同
因为所有活动完成了工程才算结束
所以所有从源点到汇点的路径中, 路径长度最大的为关键路径
完成整个工程的最短时间=关键路径长度



4. 拓扑排序

概念

关键路径



关键路径上的活动为关键活动
(a2, a5, a7)

从源点到汇点可能会有很多条路，长度(权值和)不同
因为所有活动完成了工程才算结束
所以所有从源点到汇点的路径中，路径长度最大的为关键路径
完成整个工程的最短时间=关键路径长度

4. 拓扑排序

概念

关键路径

BOK数据结构



关键路径求法

求出四个值：

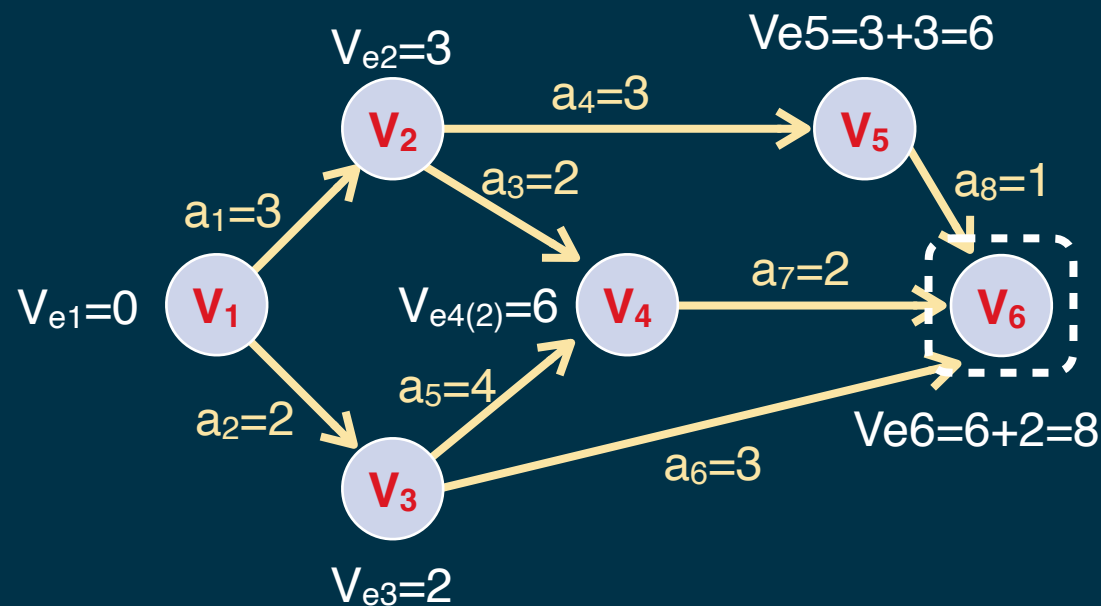
事件 v_i 的最早发生时间 v_e

事件 v_i 的最迟发生时间 v_l

活动 a_i 的最早开始时间 e

活动 a_i 的最迟开始时间 l

AOE网 (有权值)



同时有多条路径时，选择最长的作为[最早发生时间]
因为下一个事件需要前置活动全结束了才能开始

4. 拓扑排序

概念

关键路径

BOK数据结构



关键路径求法

求出四个值：

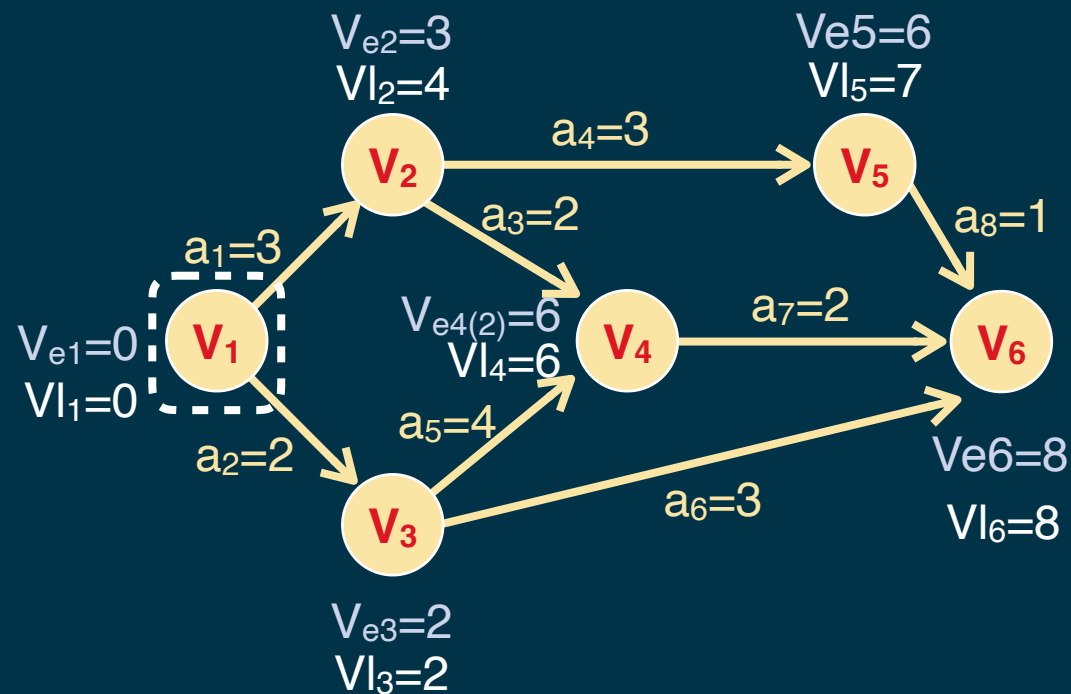
事件 v_i 的最早发生时间 v_e

事件 v_i 的最迟发生时间 v_l

活动 a_i 的最早开始时间 e

活动 a_i 的最迟开始时间 l

AOE网 (有权值)



同时有多条路径时，选择最短的作为「最迟发生时间」
因为要确保本事件在更严格的条件下也能保证项目按时完结

每个顶点（事件）的最迟开始时间要根据顺序从汇点开始倒推

4. 拓扑排序

概念

关键路径

BOK数据结构



关键路径求法

求出四个值：

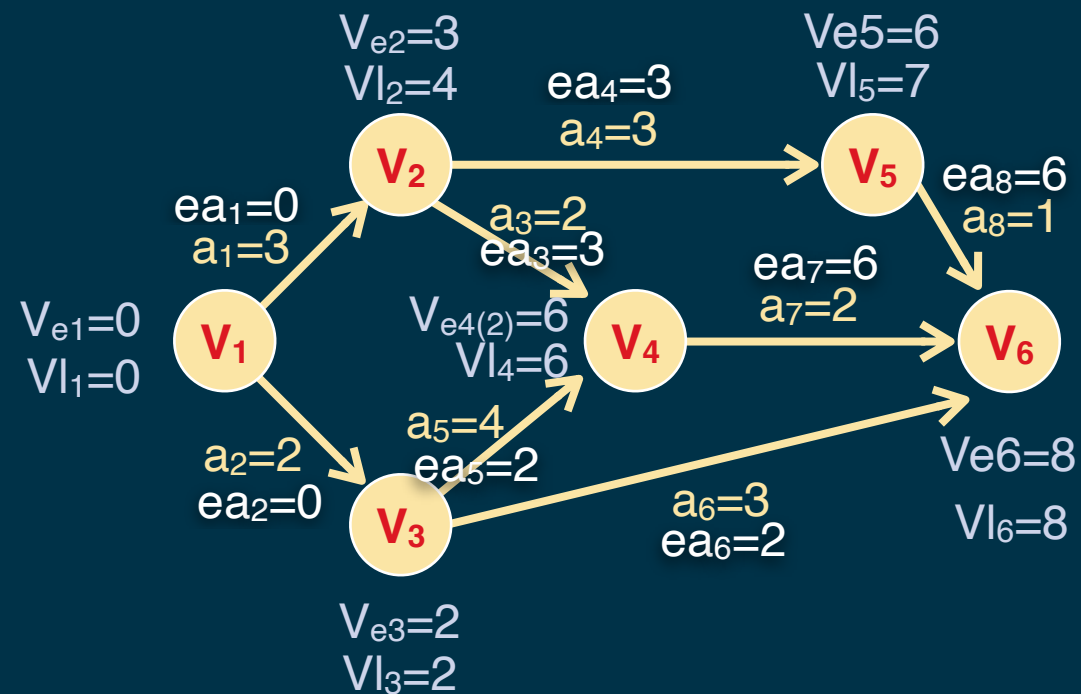
事件 v_i 的最早发生时间 v_e

事件 v_i 的最迟发生时间 v_l

活动 a_i 的最早开始时间 e

活动 a_i 的最迟开始时间 l

AOE网 (有权值)



弧 (活动) 的最早开始时间 e 等于该弧起点顶点的 v_e

4. 拓扑排序

概念

关键路径

BOK数据结构



关键路径求法

求出四个值：

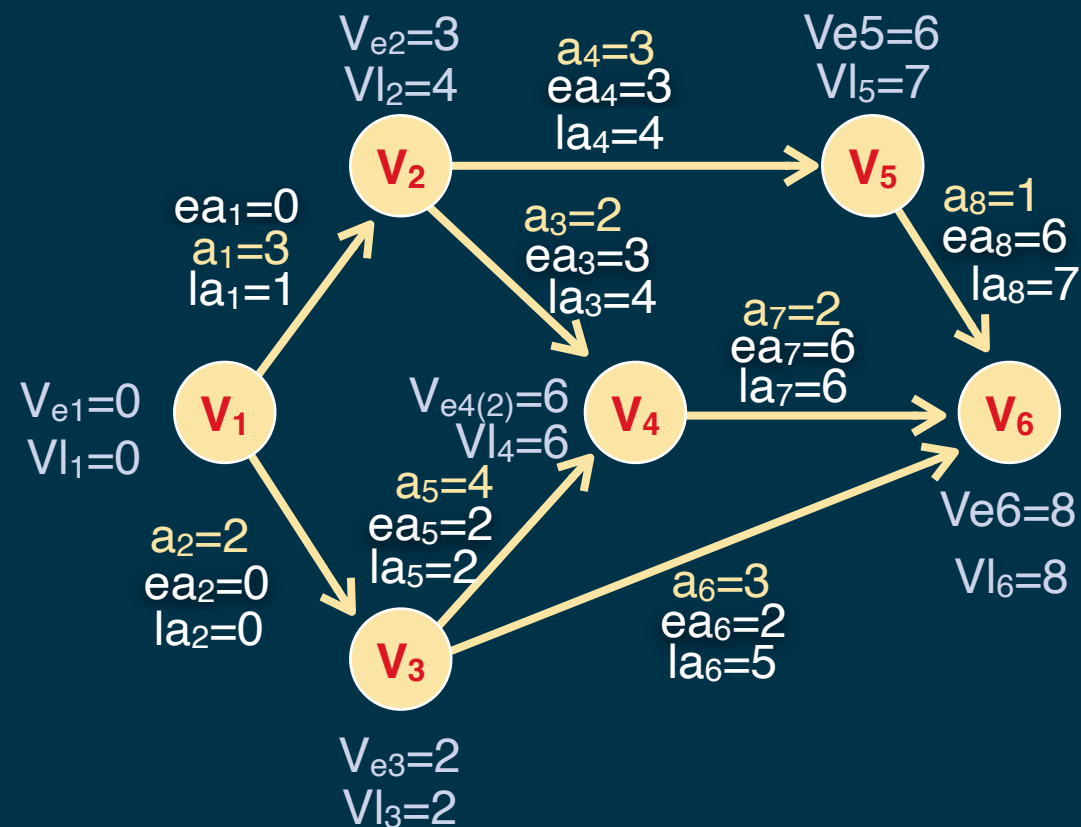
事件 v_i 的最早发生时间 v_e

事件 v_i 的最迟发生时间 v_l

活动 a_i 的最早开始时间 e

活动 a_i 的最迟开始时间 l

AOE网 (有权值)



弧（活动）的最迟开始时间 l 等于
该弧终点的顶点的 v_l 减去该弧持续的时间

关键路径求法

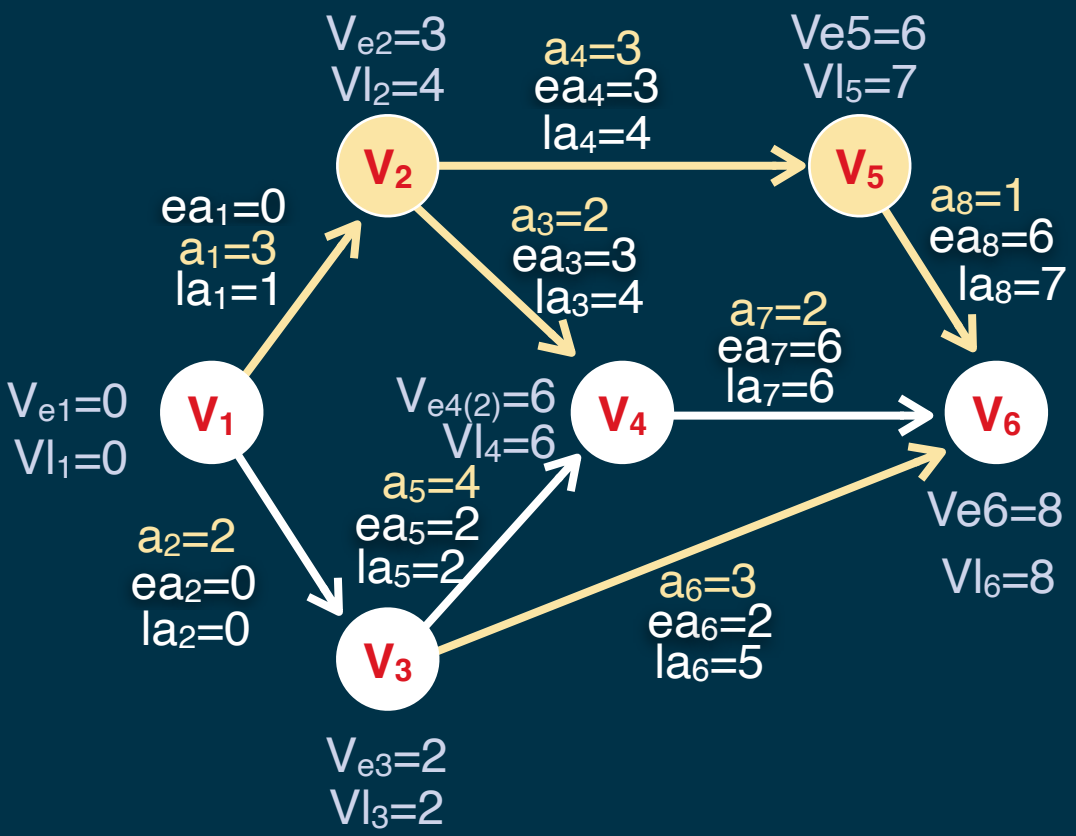
求出四个值：

- 事件 v_i 的最早发生时间 v_e
- 事件 v_i 的最迟发生时间 v_l
- 活动 a_i 的最早开始时间 e
- 活动 a_i 的最迟开始时间 l

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
$e(i)$	0	0	3	3	2	2	6	6
$l(i)$	1	0	4	4	2	5	6	7
$l(i)-e(i)$	1	0	1	1	0	3	0	1

活动的最早和最迟开始时间无任何余量
则此活动为关键活动
这些活动组合起来就是关键路径

AOE网 (有权值)





4. 拓扑排序

概念

关键路径

1. 关键路径并不唯一
2. 关键路径是权值之和最大的那条路径
3. 增加关键路径上的任一关键活动的持续时间，一定会延长整个工期
4. 缩短关键路径，并不一定导致整个工期加快
 - 如果只有一条关键路径，缩短关键路径，一定导致整个工期加快
 - 如果有多条关键路径，缩短关键路径，不一定导致整个工期加快
因为缩短了路径长度的那条关键路径可能不再是关键路径